

Each sudoku puzzle is modelled as a constraint satisfaction problem (CSP) and solved by backtracking search, a depth-first search that assigns one variable at a time (Russell and Norvig, 2022, §5.2-5.3). Search effort is reduced by inference from each assignment (forward checking, singleton domain and single-support propagation) and by a fail-first variable-ordering heuristic (Haralick and Elliott, 1980) which adapts the minimum-remaining values (MRV) (Russell and Norvig, 2022, §5.3.1) to also consider digit placement (Knuth, 2000).

The variables are the 81 cells  $\chi_{r,c}$ :  $r, c \in \{0, \dots, 8\}$  each with a domain  $D = \{1, \dots, 9\}$ . There are 27 Alldiff constraints  $C$ , one per unit (row, column and 3x3 box) which can be expanded into binary constraints between a cell and its 20 neighbours sharing a unit with it (Russell and Norvig, 2022, §5.2.6). A state is a partial assignment and a solution is a complete assignment that adheres to all constraints. Backtracking search selects an empty cell (baseline: next empty cell from scanning row by row), tries each allowed value that does not clash with a neighbour, recurses extending the guessing and backtracks when finding contradictions (no values fit). If all branches fail then a puzzle is returned as unsolvable. With  $d = 9$  and  $m$  empty cells, the baseline has  $O(9^m)$  time and  $O(m)$  space complexity (Russell and Norvig, 2022, §3.4.3 and §5.3).

Optimisations were made in stages and targeted to improve the branching factor  $b$ , depth  $p$  or cost per node. Forward checking (Russell and Norvig, 2022, §5.3.2) results in only values that are still in the cell's domain being tried, which helps to reduce  $b$  and also supports early stopping if a neighbour's domain is emptied [hard puzzles: average nodes decreased from 162,269 to 28,876 and time from 1.44 s to 0.4 s]. MRV (Russell and Norvig, 2022, §5.3) explores the cell with fewest domain options which minimizes  $b$  so dead ends are detected before additional branches are built. Hard-puzzle nodes on average dropped from 28,876 to 2,015 (14 times) and time from 401 to 27 ms (15 times). Constraint propagation after value assignment assigns singleton domains (Russell and Norvig, 2022, §5.2.5) and single-support values (a digit with one place to go in its unit), which is a partial version of generalised arc consistency on the unit's Alldiff (Dechter, 2003, pp. 70-73). Inferred cell assignment creates no nodes so  $p$  decreases and for the hard puzzles on average: guesses fell from 2,257 to 23 and nodes to 13. The cost is seen for very easy to medium puzzles, now solved with no guesses so at the clue-loading stage, their average time to solve increased from 0.11 to 0.68 ms, which is a known trade-off between forward checking and stronger propagation per node (van Beek, 2006 p.96). Stopping the support count check at 2 (since we cannot infer at this stage), was able to recover about one third of this time. Extending the single-support check to the values discarded from the cell's domain due to assignment supported additional inferences with hard puzzles nodes dropping 4 times, number of guesses dropping 5 times and time to solve by 27% on average. This version still needed 115s and 2.6 million nodes to conclude Norvig's (2006) impossible puzzle is unsolvable. This motivated the addition of digit branching (Knuth, 2000, p.12) such that the next guess is made on whichever is lower: the number of cells that can take a digit or the cell's MRV. Norvig's (2006) puzzle was now deemed unsolvable in 74 ms and 1,312 nodes. Since  $b$  remained at approximately two guesses per node for both versions, the reduction in nodes must be a result of a reduction in  $p$ . By having the option to branch on digits rather than only cell values, fail-first (Haralick and Elliott, 1980) supports earlier detection of failures. Since Norvig's puzzle is unsolvable, the full search tree  $b^p$  must be checked, so the decrease in  $p$  is compounded.

My final solver produced the correct result for the 60 provided puzzles (hard puzzles: mean 0.6 ms, 3.3 nodes, 4.4 guesses) and it was also able to solve famously hard puzzles (Inkala, Unsolvable #28 & # 49 and Escargot; Stuart, 2012) in under 33 ms. At present my optimisations focus on minimising the number of nodes checked which has been successful but I did not yet explore optimisations to make each node cheaper to process. With so few nodes being visited in the current implementation this is the main limitation. For very easy puzzles the solver needs on average 0.53 ms to solve compared to the 0.09 ms needed for the baseline. This is because each assignment, including those inferred from clue loading, triggers propagation that will repeatedly check units for removed values and each guess also needs to copy all 81 cells and their domain sets. Future work is recommended to reduce the cost per node. The first area I would investigate is adjusting the solver to store each cell domain as a 9-bit integer so that value checks and removals become single bitwise operations (Lecoutre and Vion, 2008, §3.1) and copying a state involves copying 81 integers rather than 81 sets. Taking this further I would like to explore if I could remove the need for per-guess copying. Taking inspiration from Knuth's (2000) dancing links I would aim to apply the undo idea by keeping track of each value removed from a domain following a guess and using this list to revert them when backtracking. This would mean cost per node scales with the number of values that get removed and not all 81 cells and sets each time.

## References:

- Dechter, R., 2003. *Constraint processing*. San Francisco, Calif.: Morgan Kaufmann.
- Haralick, R.M. and Elliott, G.L., 1980. Increasing tree search efficiency for constraint satisfaction problems. *Artificial intelligence*, 14(3), pp.263--313.
- Knuth, D.E., 2000. Dancing links. Stanford, Calif.: Stanford University. arXiv [Online]. Available from: <https://arxiv.org/abs/cs/0011047> [Accessed 21 September 2026].
- Lecoutre, C. and Vion, J., 2008. Enforcing arc consistency using bitwise operations. *Constraint programming letters* [Online], 2. Available from: <https://webusers.i3s.unice.fr/~regin/Cpl/Papers/v2/Vol-2-Lecoutre-Vion.pdf> [Accessed 28 September 2026].
- Norvig, P., 2006. *Solving every Sudoku puzzle* [Online]. Available from: <https://www.norvig.com/sudoku.html> [Accessed 28 September 2026].
- Russell, S.J. and Norvig, P., 2022. *Artificial intelligence: a modern approach*. 4th ed., global ed. Harlow: Pearson.
- Stuart, A., 2012. *Arto Inkala Sudoku* [Online]. SudokuWiki.org. Available from: [https://www.sudokuwiki.org/Arto\\_Inkala\\_Sudoku](https://www.sudokuwiki.org/Arto_Inkala_Sudoku) [Accessed 28 September 2026].
- van Beek, P., 2006. Backtracking search algorithms. In: F. Rossi, P. van Beek and T. Walsh, eds. *Handbook of constraint programming*. Amsterdam: Elsevier, pp.85--134.